RayLiDAR

Adaptive GPU LiDAR Simulation with Hybrid Mesh + 3D Gaussians

Sizhe Liu · Xuan Zhu · Jing Huang

One scene. Three approaches.
One fair benchmark.



Motivation

01

Real-world data collection is expensive

Physical LiDAR testing needs specialized sensors, vehicles, and repeated field runs.

02

Simulation performance is a bottleneck

Large city-scale digital twins require millions of ray queries and substantial GPU compute.

03

Different representations have different trade-offs

Triangle meshes and 3D Gaussian Splatting differ in accuracy, memory use, and runtime.

OUR GOAL

Use adaptive Mesh–Gaussian simulation to reduce GPU cost while preserving LiDAR fidelity.

CORE IDEA

One scene, Three simulation modes.

Same LiDAR pose. Same scan pattern.

01 **Mesh**
Explicit geometry · Vulkan hardware RT

02 **3D Gaussian**
Reconstructed detail · GPU sensor returns

03 **Hybrid**
Adaptive routing · Cost / accuracy aware



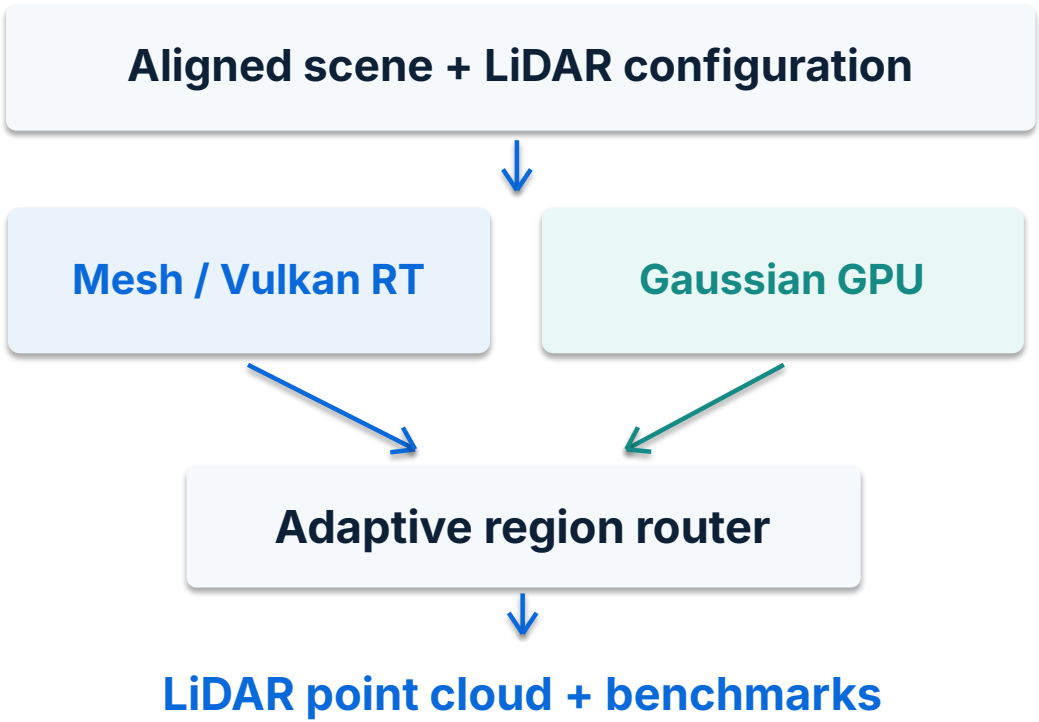
OUR HYPOTHESIS

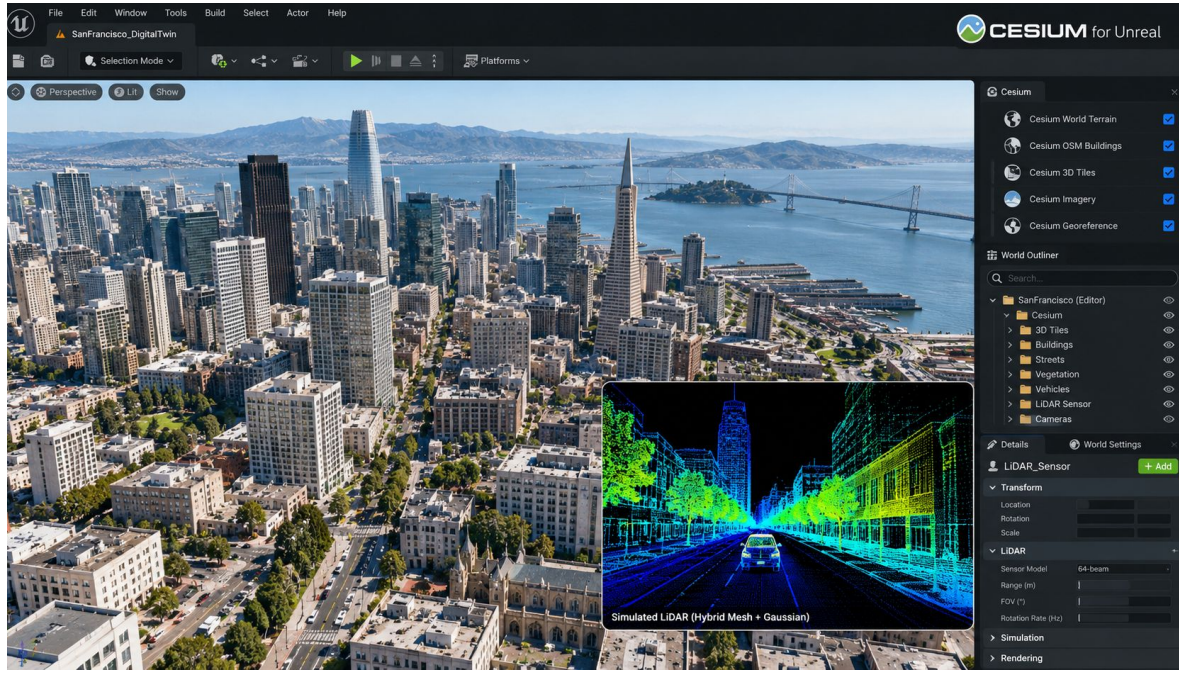
A better speed–accuracy trade-off — not a guaranteed speedup.

TECHNICAL APPROACH

GPU core + Cesium visualization.

Keep the rendering backend independent from the Unreal viewer.





UE5 + Cesium / City digital twin

VALIDATION

Measure the trade-off. Do not assume it.

Mesh-only vs. Gaussian-only vs. Adaptive Hybrid

01

Latency

ms / simulated scan

02

Memory

average / peak GPU allocation

03

Fidelity

range error + valid returns

Same scene · Same sensor settings · Same LiDAR poses

KITTI-360 validation will be used for real world validation

Three checkpoints. One final demo.

A buildable scope, with measurable deliverables.

MILESTONE 1 WEEKS 1–2	**Scene + mesh baseline**	Aligned street; LiDAR setup; Vulkan RT scans.
MILESTONE 2 WEEKS 3–4	**Gaussian + hybrid**	Gaussian baseline; region routing; unified output.
MILESTONE 3 WEEK 5	**Optimization + validation**	GPU profiling; 3-mode benchmarks; KITTI-360 comparison.
FINAL WEEK 6	**Working RayLiDAR demo**	Side-by-side visualization + accuracy / performance report.